Here is the code that handles 'read':

```
static ssize_t skel_read(struct file *file, char *buffer, size_t count, loff_t *ppos)
{
        struct usb_skel *dev;
        int retval;
        int bytes_read;

        dev = (struct usb_skel *)file->private_data;

        mutex_lock(&dev->io_mutex);
        if (!dev->interface) {          /* disconnect() was called */
                retval = -ENODEV;
                goto exit;
        }

        /* do a blocking bulk read to get data from the device */
        retval = usb_bulk_msg(dev->udev,
                              usb_rcvbulkpipe(dev->udev, dev->bulk_in_endpointAddr),
                              dev->bulk_in_buffer,
                              min(dev->bulk_in_size, count),
                              &bytes_read, 10000);

        /* if the read was successful, copy the data to userspace */
        if (!retval) {
                if (copy_to_user(buffer, dev->bulk_in_buffer, bytes_read))
                        retval = -EFAULT;
                else
                        retval = bytes_read;
        }

exit:
        mutex_unlock(&dev->io_mutex);
        return retval;
}
```

Having mentioned the drivers, let's focus our attention on the USB filesystem (USBFS). This essentially tries to make the USB devices (that you have attached to your system) work like ordinary files in Linux.

USBFS tracks all the USB devices that you are attaching and removing from the system ('bus'). You may also note that the filesystem was initially developed as an extension of the *devfs* filesystem. And USBFS is quite similar to the */proc* filesystem, if you consider the dynamic nature of both the filesystems.

An important point worth mentioning here is that we follow a set of 'rules' by convention. Though your filesystem may still work properly even if you don't follow these. For example, you can mount the filesystem anywhere, but the recommended location is */proc/bus/usb* so that the user-space utilities are not affected. Also, while mounting this filesystem, we use the following code (as the root):

```
mount -t usbfs none /proc/bus/usb
```

Here, too, you can choose your own custom name instead of 'none'; some people prefer to use 'usbdevfs' instead of 'none'. After mounting the filesystem, we can see the files listed under it:

```
linux-el8p:/home/aasisvinayak # mount -t usbfs none /proc/bus/usb
linux-el8p:/home/aasisvinayak # cd /proc/bus/usb
linux-el8p:/home/aasisvinayak # ls
001  002  003  004  005  006  007 devices
linux-el8p:/home/aasisvinayak #
```

You may note that the directories (the entries in monospace bold fonts in the above snippet) will lead you to the devices mounted. You can also find a file named *devices*, which contains many entries in the following pattern:

```
T:  Bus=08 Lev=00 Prnt=00 Port=00 Cnt=00 Dev#=  1 Spd=12  MxCh= 2
B:  Alloc=  0/900 us ( 0%), #Int=  0, #Iso=  0
D:  Ver= 1.10 Cls=09(hub  ) Sub=00 Prot=00 MxPS=64 #Cfgs=  1
P:  Vendor=1d6b ProdID=0001 Rev= 2.06
S:  Manufacturer=Linux 2.6.31.12-0.1-pae uhci_hcd
S:  Product=UHCI Host Controller
S:  SerialNumber=0000:00:1d.2
C:* #Ifs= 1 Cfg#= 1 Atr=e0 MxPwr=  0mA
I:* If#= 0 Alt= 0 #EPs= 1 Cls=09(hub  ) Sub=00 Prot=00 Driver=hub
E:  Ad=81(I) Atr=03(Int.) MxPS=   2 Ivl=255ms
```

These entries correspond to a list of all your USB devices attached to your system. And here are the explanations of the code letters used in the file:

- T – Topology related aspects
- B – Bandwidth specific details
- D – Device descriptor information
- P – Product ID and vendor information
- S – For adding any 'string descriptors' (see the code)
- C – Carries descriptions concerning configuration
- I – Interface descriptions
- E – Endpoint descriptions

Coming back to the directory structure, you may have noticed that it follows this pattern: */proc/bus/usb/BBB/*. Here 'BBB' is the number of the USB bus. And this number is used by the application for communicating with the USB device. As you can see in the above terminal snippet, the name starts from 001 and goes up to the total number of devices.

Meddling with USB devices

If you try to read the device file, the code will return the raw USB descriptor. This includes the device descriptor (D) and the configuration descriptors (C). You can use *ioctl* calls if you want to send or receive data. Here is the structure of *usbdevfs_ioctl*:

```
struct usbdevfs_ioctl {
        int     ifno;           /* interface 0..N ; negative numbers reserved */
        int     ioctl_code;     /* MUST encode size + direction of data so the
                                 * macros in <asm/ioctl.h> give correct values */
        void __user *data;      /* param buffer (in, or out) */
};
```